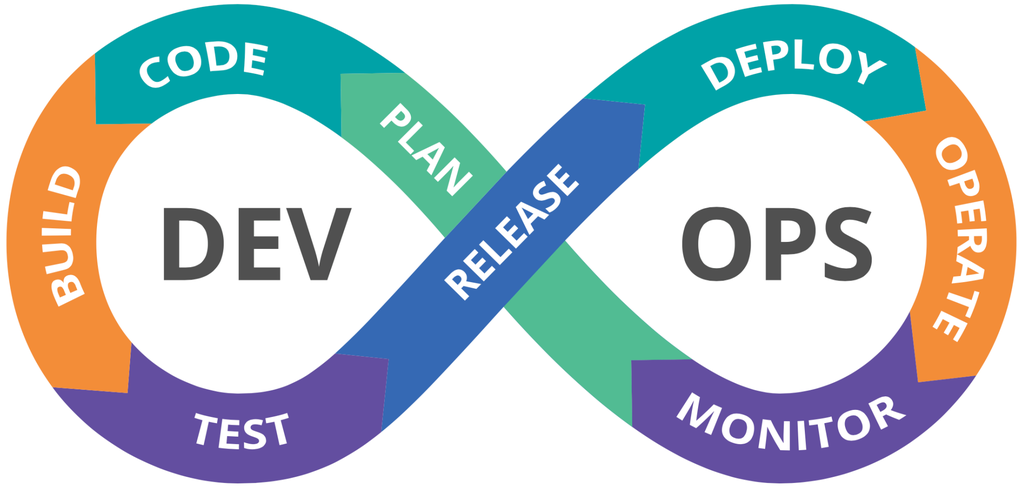
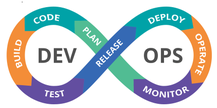


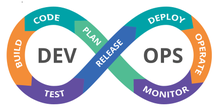
Kratak uvod

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis



DevOps?

- Kovanica koja se često čuje, ali šta bi trebalo da znači?
  - Dev - Development
  - Ops - Operations (IT operations)
    - U tradicionalnom procesu razvoja softvera, programeri razvijaju aplikaciju i unapređuju je kada se za to javi potreba, dok (IT) operativni tim brine da su aplikacije korektno instalirane, održavane i nadgledane kako bi obavljale ono za šta su namenjene. Ova dva procesa (razvoj i produkciona upotreba aplikacije) i tima su u ovakvoj postavci prilično izolovani.

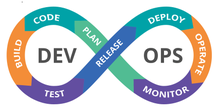


DevOps?

- Izraz DevOps se pojavio 2009 (skovao ga je Patrick Debois), kao naziv koji sugeriše suštinu pristupa koji objedinjuje procese razvojnog tima sa procesima operativnih timovima

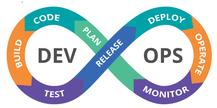
- Ali DevOps nije sam po sebi neki propisani proces, standard ili određena tehnologija, već “kultura” ili jedan (integrisani) način pristupa razvoju softvera

- Gartner: “DevOps predstavlja promenu kulture u IT-u, i to takvu da se fokusira na brzu isporuku IT usluga, usvajanjem agilnih i optimizovanih praksi (*lean practices)* u kontekstu sistemski orijentisanog pristupa. DevOps naglašava ulogu ljudi (i kulture rada) i teži da unapredi saradnju između operativnih timova i razvojnih timova. DevOps pristup značajno se oslanja na korišćenje dostupne tehnologije - posebno automatizovane alate koji mogu da iskoriste nove trendove u kojima je, iz perspektive životnog ciklusa razvoja (softvera), i sama infrastruktura postala dinamička i programabilna.”



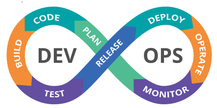
DevOps?

- Dakle ključni za uspeh DevOps-a su:
  - **ljudi (organizacija)**
  - **procesi**
  - **alati**



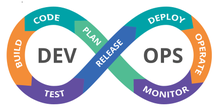
DevOps organizacija

- U DevOps-u, izolovanje razvojnih i operativnih timova nestaje. Ali cilj nije samo da ovi timovi počnu blisko da sarađuju, već da se uspostavi takvo okruženje u kojem članovi razvojnog tima u svakoj fazi razvoja proizvoda (softvera) rade zajedno sa operativnim timovima (koji imaju najveći uvid u očekivanja krajnjih korisnika i kako se aplikacija zaista koristi). Drugim rečima, sama distinkcija između ovih timova bi trebala da iščezne i da se oni uvek posmatraju kao jedan integralni DevOps tim.



DevOps procesi

- Kako bi se obezbedilo da ponavljajući manuelni zadaci ne usporavaju razvoj i isporuku proizvoda, DevOps procesi teže da se baziraju u maksimalno mogućoj meri na konceptima “infrastructure as code” što omogućava timovima da automatizuju mnoge korake u životnom ciklusu razvoja i produkcione upotrebe softverskih proizvoda - obezbeđivanje i upravljanje neophodnim infrastrukturnim resursima. Primer: (instalacija i konfigurisnje servera).



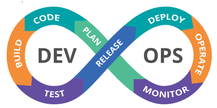
DevOps alati

- Osnovni pokretač uspešne primene DevOps principa je upotrebna alata koji omogućavaju automatizaciju i upravljanje određenim fazama u životnom ciklusu razvoja proizvoda (softvera)

- Zrelost tehnologija virtuelizacije, računarstva u oblaku i pojava IaaS su obezbedili dovoljno fleksibilnu, efikasnu i brzu platformu koju DevOps zahteva. Danas se uobičajeno spominju Docker, Kubernetes, GitHub, Jenkins, Travis CI … alati koji se koriste u DevOps. Ali da li su oni suština DevOps?

  - NE, oni bi se podjednako dobro mogli koristiti i u tradicionalnom pristupu, ključno je kako se neki alat koristi. Ne postoji skup predefinisanih alata koji su “DevOps alati” već postoje različiti alati za različite namene koji mogu biti korišćeni u kontekstu DevOps.

  - Istovremeno i kontejnerizacija i kontrola izvornog koda i kontinualna integracija (sve ono za šta se koriste gore pomenuti alati) JESU operacije koje imaju veliku ulogu u DevOps kontekstu.

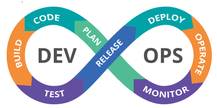


DevOps koreni

- DevOps nije nastao odjednom i ni iz čega, dva primarna područja koja su prethodnici DevOps su:

- **Enterprise systems management (ESM)**. Sistem administratori su jedno od zanimanja koje je izvršilo jak uticaj na definisanje DevOps koncepta. Njihovo iskustvo i duboko poznavanje ESM je donelo neke ključne koncepte u DevOps - upravljanje konfiguracijama, nadzor sistema, automatizacija pribavljanja potrebnih konfiguracija, kao i pristup baziran na automatizovanoj primeni različitih alata u programiranom redosledu.

- **Agile development.** Agilni razvoj softvera je definisao neophodnost bliske saradnje između klijenata, product management-a, razvojnih timova, QA kako bi se prevazišle slabe tačke u komunikaciji i omogućio koncept brzih razvojnih iteracija koje dovode do poboljšanja proizvoda. DevOps u potpunosti prihvata ove koncepte, ali ih nadograđuje primenjujući filozofiju da je i način isporuke servisa, konfigurisanja aplikacija i sistema od ključne važnosti za klijenta i samim tim moraju biti među primarnim problemima koji se razmatraju, a komunikacija sa operativnim timovima isto tako važna.

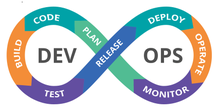


Problemi koji su doveli do pojave DevOps-a

- Razvojni timovi i sistem administratori dosta često mogu imati neusaglašeno dejstvo, primarno uzrokovane čestim novim zahtevima krajnjih korisnika - koji žele nove servise, nove funkcionalnosti što je brže moguće. Ali istovremeno klijenti očekuju stabilno okruženje (bez otkaza i prekida u isporuci servisa).

- Ovo je često dovodilo do krucijalne dileme - obezbediti brze izmene (što razvojni timovi mogu da obezbede) ne vodeći mnogo računa o uticaju promena na stabilnost, ili usporiti razvoj kako bi se potvrdila stabilnost sistema.

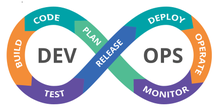
- Nijedna od ovih krajnosti na kraju nije zadovoljavajuća za krajnjeg klijenta.



Problemi koji su doveli do pojave DevOps-a

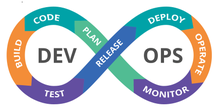
- DevOps je osmišljen upravo sa ciljem da objedini sve koji su povezani sa razvojem softvera i njegovim puštanjem, održavanjem i korišćenjem u produkcionom okruženju - poslovne korisnike, razvojne timove, test inženjere, sistem administratore

- Oni se integrišu u (što je moguće više) automatizovani radni proces čiji je primarni cilj obezbediti brzu isporuku visokokvalitetnih softverskih rešenja koja zadovoljavaju korisničke zahteve, istovremeno čuvajući integritet i stabilnost celokupnog isporučenog rešenja.



Problemi koji su doveli do pojave DevOps-a

- Ovaj cilj se postiže konsenzusom oko određenih zajedničkih principa
  - utvrđivanje očekivanja i prioriteta
  - saradnja unutar timova i između timova
  - automatizacija uobičajenih i ponavljajućih aktivnosti, kako bi se oslobodilo vreme za složenije zadatke
  - integracija povratnih informacija iz produkcije, koristeći se nekim merljivim pokazateljima iz produkcije
  - deljenje podataka među svim zainteresovanim kako bi se obezbedila što bolja saradnja



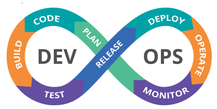
Odnos DevOps-a i drugih često pominjanih pristupa

- Agilne i optimizovane metode razvoja definišu kako timovi sprovode razvojne cikluse kako bi se obezbedio brz odgovor na nove zahteve. Koncentirše se na razvoj “kulture” rada, alatima koje će timovi koristiti se ne bavi.

- DevOps - opisuje kako organizacije sarađuju - oslanjajući se na timove koji prevazilaze “granice” određenih specijalizacija. DevOps polazi od “kulture” rada i uvodi alate koji potpomažu usvojene principe rada.

- SRE (System Reliability Engineering) - opisuje kako se određene aktivnosti automatizuju, pri čemu se određene zahtevne operacije dodeljuju ekspertima. U ovom slučaju kreće se od korišćenja alata ka uvođenju principa rada.

- DevOps podvarijante (npr. SecDevOps) - opisuje kako određene specifične prakse treba ugraditi u životni ciklus razvoja softvera u što ranijoj fazi kako vi bi se u sam softver ugradili određene poželjne osobine.

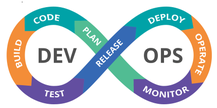


Funkcionisanje DevOps-a

- Iako postoje različite prakse, sve u opštem slučaju počivaju na sledećim osnovama

- **Saradnja (Collaboration)** - razvojni timovi i operativni timovi rade zajedno, konsultujući se na vreme i koordinišući aktivnosti - umesto da traže krivicu za određene probleme u nečemu što je drugi tim uradio

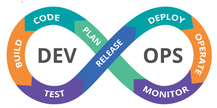
- **Automatizacija** - uspeh DevOps-a u velikoj meri počiva na automatizaciji, tj. upotrebi alata koji omogućavaju da se određene faze razvojnog ciklusa automatizuju. Pri tome upotreba pojedinačnih alata nije proizvoljna i po nasumičnom redosledu, već se dizajnira konkretan sled korišćenja alata (toolchain).



Funkcionisanje DevOps-a

- **Kontinualna integracija** - jedan od temelja DevOps-a je kontinualna integracija - tehnika koju je dizajnirao i imenovao Grady Booch - izmene izvornog koda koje obavljaju različiti programeri konstantno se (automatizovano) spajaju u zajednički glavnu razvojnu granu. Ovo sprečava da pojedinačna razvojna grana kod jednog programera divergira predaleko od ostalih i time se izbegavaju katastrofalni konflikti prilikom spajanja koda.

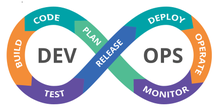
  - Ovaj pristup “tera” programere da često spajaju svoj kod sa ostatkom tima, čime se potencijalni problemi integracije uočavaju rano. Kako bi ovo funkcionisalo neophodna je i češća komunikacija između članova tima - čime se dodatno promoviše saradnja unutar tima.



Funkcionisanje DevOps-a

- **Kontinualno testiranje** - testiranje počinje već na strani razvojnog tima. Testira se sve i konstantno iznova. Time se izbegava mogućnost da se isporuči proizvod koji sadrži greške koje katastrofalno deluju na upotrebu softverskog rešenja. Operativni timovi imaju već ovde veliku ulogu, jer mogu obezbediti korektnu konfiguraciju platformi za testiranje, ali i proveriti da li su odgovarajući alati za nadzor korektno konfigurisani kako bi se pravilno očitala određena stanja u aplikaciji.

  - integrisano testiranje omogućava razvojnim timovima da postignu odgovarajući balans između brzine razvoja i kvaliteta isporučenog softvera.

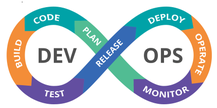


Funkcionisanje DevOps-a

- **Kontinualna isporuka (continuous delivery)** - predstavlja praksu razvoja kod koje se izmene u kodu automatski “bilduju”, testiraju i pripremaju (pakuju) za isporuku u produkciono okruženje.

  - Ovaj princip se nadovezuje na kontinualnu integraciju, pri čemu se kod koji je integrisan automatski instalira na test okruženje i/ili produkciono okruženje nakon uspešnog bildovanja (i testiranja).

  - kada se ova faza korektno implementira razvojni timovi uvek imaju na raspolaganju barem jednu verziju koja je prošla standardizovani proces i koja je u svakom momentu spremna za instaciju (isporuku).



Funkcionisanje DevOps-a

- **Kontinuirani nadzor** - obebeđuje informaciju o ponašanju isporučenog softvera. Mere se performanse i dostupnost (availability) softverskog rešenja kako bi se na osnovu ovih podataka poboljšala stabilnost rešenja.

  - Kontinuirani nadzor (monitoring) pomaže da se lakše otkriju ključni uzroci odrešenih problema kako bi se proaktivno izbegli otkazi i umanjili efekti takvih stanja na krajnjeg korisnika. Neki eksperti čak insistiraju da monitroing mora biti sastavni deo definicije bilo kog servisa.